

Intersection Light Control

Embedded Systems, IS1300

Frans Ekberg

20/12 2025

Contents

1	Introduction	3
2	Hardware	4
2.1	Configuration	5
3	Software	6
3.1	Layout	6
3.2	Program and Code	6
3.2.1	Definitions and global variables	6
3.2.2	Updating lights via SPI	7
3.2.3	Setting the Traffic Lights	8
3.2.4	Cycling the intersection direction	8
3.2.5	Interrupts	8
3.2.6	Blinking Pedestrian Indicators	9
3.2.7	Timing	9
3.2.8	Intersection Control	10
3.3	Testing	11
3.3.1	setLightsTest	11
3.3.2	Interrupt Tests	11
3.3.3	Timing Tests	11
3.3.4	Logic Tests	12
4	Summary and Discussion	13
5	References	14
6	Appendix	15

1 Introduction

This report will cover the process of making a traffic light controller as well as configuring the hardware and software needed for the program to run according to some predefined specifications. The hardware and the specifications are provided in the course Embedded Systems IS1300 at KTH.

The hardware provided is designed to have the same functionality as a traffic intersection, with traffic lights for cars in four directions and for pedestrians on two crosswalks. The hardware will be covered more thoroughly in section 2.

The specifications are provided in five separate implementation tasks. Only the first three will be covered in this report. Essentially these specifications state that the lights in the intersection should be controlled in such a way that; no two cars that have conflicting paths can ever have a green light at the same time and that a pedestrian cannot have a green light at the same time as the car lane they are crossing has a green light.

2 Hardware

The hardware used in this project consists of two separate boards. The hardware that stores the program and runs the code is a STM32 NUCLEO-L476RG board [1]. This board contains some modules that are used and need to be configured, see section 2.1 for more details. This board uses a STM32L476RG Microprocessor to control all modules on the board.

The other provided hardware is the Traffic Light Shield [2]. This board has no Microprocessor and has to be controlled by the NUCLEO board. The Traffic Shield can be connected to the NUCLEO board using pins on the Traffic Shield, see *Figure 1*. The Traffic Shield contains several modules that are used in this project. Three chained 74HC595D shift registers [3] control all Traffic light LEDs on the Shield. Four switches, one in each lane on the shield, are used to indicate that a car is present in that lane. Four buttons, two on each side of the pedestrian crossing, can be used by pedestrians to indicate that they want to use the crossing.

Both boards have many other modules than the ones which are shown and discussed in this project, but these will be left out of diagrams and discussion, as they are not used have no impact on the project. All used Hardware modules and how they interact can be seen in *Figure 2*

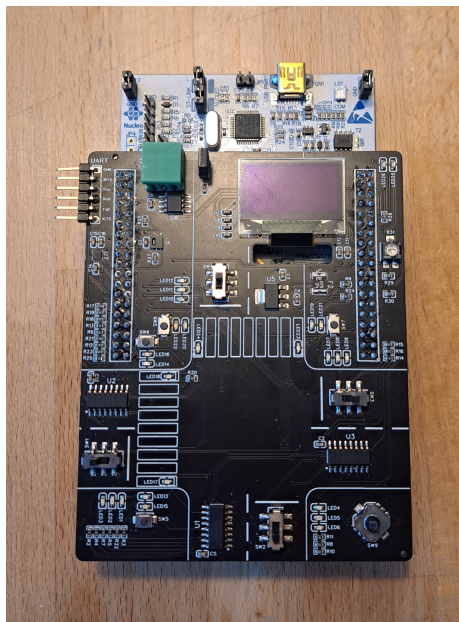


Figure 1: The STM32 L476 NUCLEO board with the Traffic Shield attached ontop.

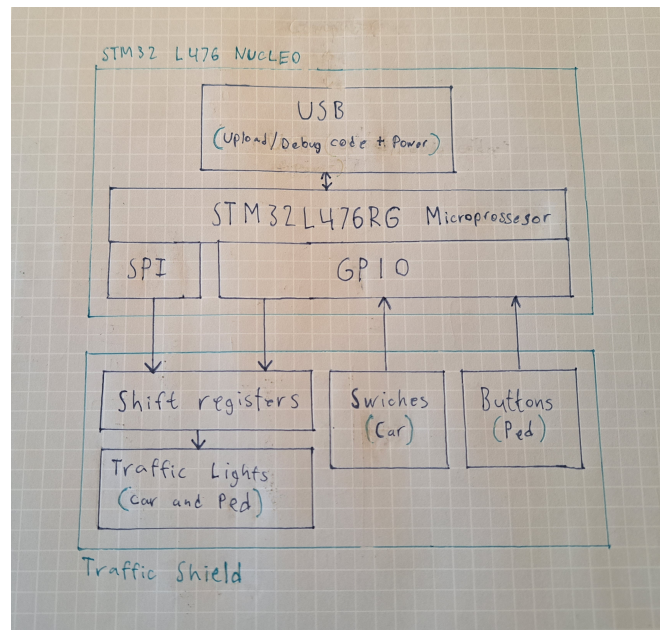


Figure 2: A Block diagram of the Hardware

2.1 Configuration

The different hardware modules on the NUCLEO board need to be configured to work as intended. These configurations were made using the STM32CubeIDE [4]. SPI is used to send several bits of data in series to the chained shift registers. GPIO signals are also used to configure the chained shift registers to work as intended. This makes it possible to control all 24 traffic light LEDs on the shield using only 5 ports. The SPI clock had to be reduced to a lower rate and the bit sent had to be set to 8. This was done to make the traffic light update in the right way.

GPIO Interrupts had to be turned on for buttons, switches and an internal timer. The buttons had to be configured to Pull-up mode, as the buttons connect to ground when pushed. The switches had to be configured to trigger on both rising and falling edge, why this is needed is explained later in section 3. The project needed to be able to time some events, for this, the module TIM3 was used. It was configured to use the internal 80 kHz timer as a clock source, and to send an interrupt every 10 ms [5].

3 Software

3.1 Layout

The software is built from the bottom up, starting at the communications with the hardware layer and moving up until the program layer is reached, where the program can be written without worrying about the hardware aspects. The general layout of the software can be seen in *Figure 3* below. What the different parts of the software do will be explained in section 3.2.

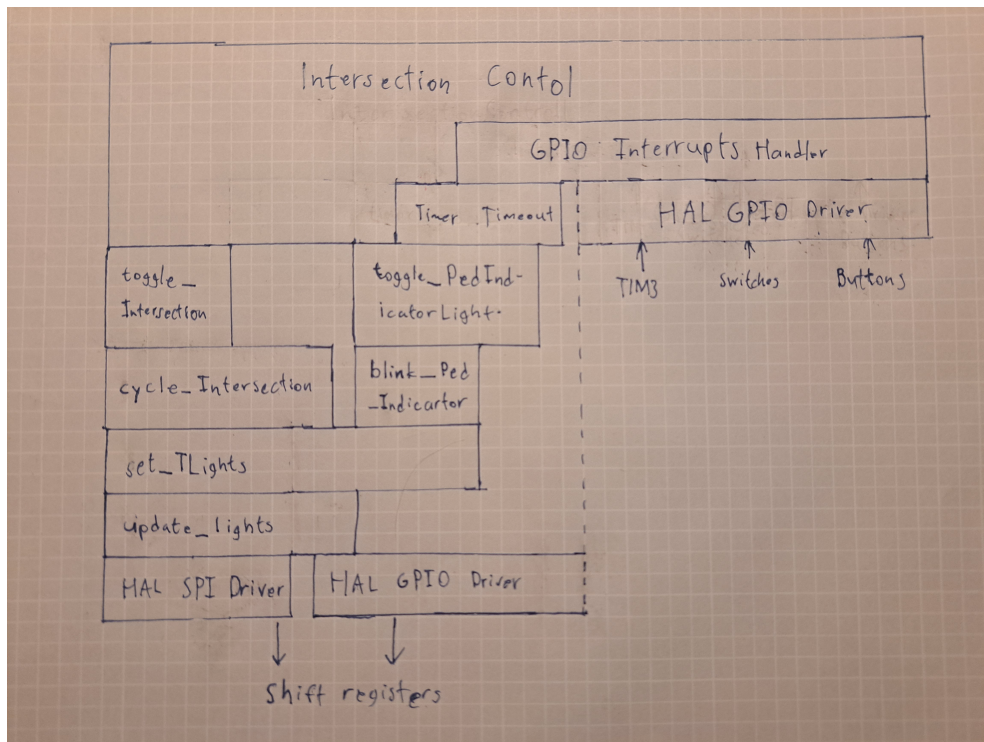


Figure 3: A Block diagram of the Software

3.2 Program and Code

3.2.1 Definitions and global variables

To make the code easier to read and the development process easier, some definitions were used to refer to integers. Some of these definitions were designed with the hardware in mind, more specifically the output of LEDs shift registers. Some examples are that the first shift register in the chain contained the LEDs for the traffic light that cars coming from the left/west of the intersection. Therefore the definition `"#define WLight 0"` was chosen. The same logic was used in defining the rest of the directions in

the intersection.

The orientation of the boards seen in *Figure 1* is the orientations used to define direction in reference to the intersection throughout the project.

The different phases that the traffic lights were defined using the bits corresponding to the correct LEDs being lit up. For example, the bit mask of one traffic light consist of three bits where the least significant bit corresponds to the red LED of that traffic light and then, in rising bit order, yellow then green. This means that red was defined as 1, corresponding to the bit map 001. Yellow was defined as 2, 010, and green as 4, 100. The phase RedYellow was also defined as 3, 011. How these definitions where used will be explained later in section 3.2.3

The different delays used throughout the program to keep track of timing where also defined.

Some global variables were crated, most notably a variable to keep track of the current time since the intersection last change direction. Other global variables include variables for; which current direction of the intersection that had a green light; which directions cars where present in; if a pedestrian button had been pushed; if a pedestrian indicator light was on and timestamps for how long ago cars or pedestrians started waiting at the intersection.

These global variables and definitions were used by many different parts of the software and will be referenced during many of the later subsection in this section.

3.2.2 Updating lights via SPI

The code that links the Hardware with the software consists of small parts, both in initialization and during runtime. The code run during runtime resides in the function `update_lights`. The local variables `currentLights`, `nextLights` and `interruptLights` consist of a bitmask corresponding to all 24 individual LEDs in the intersection. It writes either `nextLights` or `interruptLights` into `currentLights` depending on the source of the call and uses `currentLights` as the data to be sent.

The way data is sent though the SPI, the first data sent ends up in the last shift register. The bitmask was set up in the other way, with the most significant bits corresponding to the last shift register (more details in section 3.2.3). This ment that the data had to be reversed before sending it. This was done using an array with the 8 bits corresponding to each shift register and sent after one another. The function call used to transmit the data was `HAL_SPI_Transmit`. This function transmits data without waiting for any other functions that might transmit at the same time, but as there is no other functions that written to transmit, that is not a problem.

3.2.3 Setting the Traffic Lights

The function `set_TLight` is responsible for setting a traffic light to display a color. It utilizes the bitmasks `nextLights` to do this. This function takes in a traffic light as and a color, using the definition introduced in section 3.2.1. It converts these definitions into positions in the bitmask and updates `nextLights` accordingly. The bitmask consists of 24 bits, where the least significant 8 bits correspond to the 8 output-bits of the first shift register, the next 8 bits correspond to the second and the 8 most significant bits correspond to the third.

3.2.4 Cycling the intersection direction

Reviewing the given specifications of the project, it is stated that no cars are allowed to turn left and cars are allowed to turn right on pedestrian crossings that have a green light. From the given information it was concluded that only two cycles were needed to give every direction a green light at least once. The first phase is where cars from the north, cars from the south and pedestrians crossing over the East/West road have a green light. The second phase is where cars from the East, cars from the West and pedestrians crossing the North/South road have a green light. These two directions were called NS and EW respectively.

The function `cycleIntersection` takes in a variable directions, this variable is the direction that will have green lights when the function is done. The function changes the lights in three states, with an `orangeDelay` wait time between each state. First it sets all green car lights to yellow. After a delay, it sets all yellow lights to red and all red lights to red-yellow, during this state it also switches red pedestrian lights to green and vice versa. During the final state it switches red-yellow lights to green and turns off any pedestrian indicators that might have been on before the pedestrian crossing turned green.

The use of a red-yellow state is the standard in Sweden. This is used to signal to drivers that the light is turning green. This avoids the confusion of if the yellow light indicates that it's time to stop or soon time to go.

The fact that there are only two states the intersection can be in means that a toggle can be created and the two states can be toggle through by a function. That function is called `toggleIntersection` and keeps track of the current direction of the intersection, switching to the other as it is called. This function also does more to reset timing elements, but that will be covered in section 3.2.7

3.2.5 Interrupts

GPIO Interrupts are used to keep track of cars, pedestrians and timings in the intersection. When the timer interrupts (see section 2.1) it is handled by the `TIM3_IRQHandler`, that calls the function `timer10ms_timeout`. This function does two things. It increments

the global variable `currentWaitTime` by 10, signifying that 10 ms have passed (see section 3.2.7). It also checks if a pedestrian button has been pushed. If that is the case, it checks how long ago the pedestrian indicator light toggled, and if its longer or equal to `toggleFreq`, it calls the function `toggle_PedIndicatorLight` (see section 3.2.6).

The button and switch interrupts are handled by `HAL_GPIO_EXTI_IRQHandler`. This clear the interrupt and calls the function `HAL_GPIO_EXTI_Callback`. This function does nothing, but it is declared as weak. This means a functions can be written with the same name, and this overwrites the standard, weak function. In the new `HAL_GPIO_EXTI_Callback` function written for this project, all interrupts are handle as follows:

- Interrupts form car switches toggles the cars state in the intersection, where 0 is no cars present and 1 is cars in the lane. It also reads the current wait time into the time stamp `redWaitTimeStamp` (see section 3.2.7). This is only done if the car is active while the direction of the intersection is red for that car, and if the time stamp hasn't been set already.
- Interrupts form pedestrian buttons are only recognized if the current directions gives the pedestrians a red light, as pressing the button while having a green light should not do anything. In that case two time stamps will be set, `pedWaitTimeStamp` and `ILightToggleTimeStamp` (see section 3.2.7). This will only be done if `pedWaitTimeStamp` hasn't been set already, which would mean the button has already been pressed. Then the function `toggle_PedIndicatorLight` will be run (see section 3.2.6).

3.2.6 Blinking Pedestrian Indicators

Blinking the pedestrian Indicators is done using multiple functions. The function closest to hardware is `blink_Ped_Indicator`. It simply turns a chosen indicator on or off by calling the function `set_TLight` (section 3.2.3) for the given indicator light.

The function `toggle_PedIndicatorLight` keeps track of the current state of the indicator light and toggles between ON and OFF by calling `blink_Ped_Indicator` to be turned on or off.

The function `toggle_PedIndicatorLight` is called by the timer timeout function (see section 3.2.5) after the time specified by the defined value `toggleFreq`.

3.2.7 Timing

All timing elements keep track of time using the global variable `currentWaitTime`. This variable is updated every 10 ms by the timer interrupt (see section 3.2.5).

The project mostly uses time stamps to keep track of different events. These are the timestamps used in the project and their description:

- `redWaitTimeStamp` - Set at the time the first car arrives to a red light, reset before intersection change
- `pedWaitTimeStamp` - Set at the time the pedestrian button is pressed for the first time of a red pedestrian crossing, reset after intersection change
- `ILightToggleTimeStamp` - Set every time the indicator light of a pedestrian crossing is toggled, resets after intersection change

Most timing calculations and comparisons are done in the function that controls the intersection (see section 3.2.8) or in the interrupts handler (see section 3.2.5). All timing variables, including `currentWaitTime` are reset in the function `toggle_Intersection` (see section 3.2.4). This is done to make new timestamps relevant and for the intersection to function properly. Some variables are reset before the intersection changes, for example `redWaitTimeStamp`. Others are reset after, for example `currentWaitTime`. After the intersection is done, the state of cars and pedestrians are checked again, to see if cars are still present in lanes with red light or if a pedestrian pressed the button during the intersection change. If so the relevant time stamps are set again.

3.2.8 Intersection Control

The function that controls the intersection is called "intersection". It contains a while true loop that constantly runs the functions inside. The first two elements inside the loop limit how fast the intersection can cycle using while loops.

The first one runs an empty while loop as long as the `currentWaitTime` is lower than `walkingDelay`, excluding the time the crosswalk stays green during the intersection cycle (2 times `orangeDelay`). This loop prevents the intersection from cycling while pedestrians are crossing the road. The second empty while loop runs as long as `currentWaitTime` is lower than `minGreen`. This is the minimum time that a car traffic light has to stay green, to avoid the light switching from red-yellow directly to yellow, confusing drivers.

After the limiting loops are done there are 4 scenarios that might prompt the interchange to cycle:

- **No cars in active lanes**
If there are no cars in the active lanes and there are cars in one of the other lanes, the intersection will toggle.
- **redDelayMax is exceeded**
If the time a car has been waiting at a red light exceeds the `redDelayMax`, including the time it takes for the intersection to switch to green for that car (2 times `orangeDelay`), the intersection will toggle.
- **pedestrianDelay is exceeded**
If the time since a pedestrian pressed the button exceeds the `pedestrianDelay`,

including the time it takes for the intersection to switch to green for that pedestrian (one `orangeDelay`), the intersection will toggle.

- **Sensor-less operation**

If there is no traffic in the intersection and the time since the intersection toggled (`currentWaitTime`) exceeds the `greenDelay`, including the time it takes for the intersection to switch to green for the other direction (2 times `orangeDelay`), the intersection will toggle. This is essentially how the intersection would have to work if no sensors were installed.

If one of these criteria are met, the function `toggle_intersection` is called, which resets all timings and changes the direction of the intersection. Then the program starts from the top of the intersection function again and continues forever.

If cars are coming from a direction that has a green light and no other cars or pedestrians ever come to the intersection, the direction of the intersection will never change. This is of course intended behavior, but a little unrealistic, as cars would have to be constantly coming from direction that have a green light, and from no other direction.

3.3 Testing

Testing was mostly done using STM32CubeIDE [4] debugger on the board and visually confirming that the right LEDs lit up in the right order. Early on in the process, when the board hadn't been properly configured yet, tests were made to run on the computer using c-code.

3.3.1 `setLightsTest`

A test for checking if the `set_TLights` function produced the correct outputs was implemented. This test was run a few times during development to ensure that `set_TLights` still worked as intended even after adding additional functionality. For example after adding toggling to pedestrian lights.

3.3.2 Interrupt Tests

Breakpoints were used to confirm that interrupts from buttons, switches and the timer worked. Buttons and switches were pressed and were confirmed to change the correct variables.

3.3.3 Timing Tests

The calculations for the timer interrupts were run a few times to ensure that the timer interrupted each 10 ms. The timing of the intersection was tested by causing some interrupt that cause a time stamp and then checking that all variables had the correct values. Time was also measured using an external timer between breakpoints pauses.

3.3.4 Logic Tests

Testing the overarching logic of the program was time consuming. All edge cases had to be tested many times during the development process. Testing was done by putting breakpoints in intended spots the program should go to, and if that breakpoint was not triggered, the new breakpoints were set at places where the program shouldn't go, until the problem was identified and fixed. The board was also observed to check that the logic made the traffic lights change as intended.

4 Summary and Discussion

A functioning intersection using a set of provided specifications as guidelines was created. The program lets traffic flow through the intersection in four directions while avoiding putting traffic on colliding paths. The program also takes into account traffic sensors, timing restrictions and pedestrian push buttons, making changes to the intersection in real time. With all this in mind I would say that the project is a success.

There are things that could be improved upon. Testing could be further automated and code could be more cleanly divided into more separate files. One big thing that would make the code easier to edit and more modular would be to use a RTOS (Real Time Operating System) to control the different parts of the intersection and handle timing. The reason this wasn't implemented was due to time constraints, and the fact that big parts of the program would have to be rewritten, as this was discussed while some parts of the program were already done.

If I did the project again, I think it would be good to implement some type of time system where for example, a function returns its execution time. For example, if `cycle_intersection` returned its own execution time, that could be used to calculate the maximum wait time for cars and pedestrians, instead of having to know the internal delays.

I would have liked to implement some features to make the traffic light more realistic. These features would however, with the current specifications, be invalid. One feature I wanted to add was keeping pedestrian light red a bit longer, to give cars that ran a yellow light time to vacate the intersection.

One other feature that would be interesting to explore would be an initial state where all traffic lights were red. Traffic lights would turn green based on the cars or pedestrians wanting to go through it. The good thing about this approach is that there would be very low wait times for cars approaching the intersection or pedestrians wanting to cross. I have seen this system being used on real sensor controlled intersections in Sweden. However, it works best when traffic volumes are low, and when sensors are working as intended. One problem that I have experienced with these types of intersections is that small vehicles, such as mopeds, sometimes are not detected by sensors and have to wait until a car shows up to get a green light. This wait can be very long if the traffic volume is low, at nighttime for example. It would still be interesting to explore such an intersection.

5 References

- [1] STM32 NUCLEO-L476RG, <https://www.st.com/en/evaluation-tools/nucleo-l476rg.html#documentation>, Accessed: 20 December 2025, STMicroelectronics
- [2] Matthias Becker, *IS1300-TrafficLight-Schematics*, Rev 0.1 , Link: <https://canvas.kth.se/courses/57494/files/9587521?wrap=1>, 15 December 2021, KTH IS1300
- [3] *74HC595; 74HCT595 8-bit serial-in, serial or parallel-out shift register with output latches; 3-state*, Rev 12 , Link: https://assets.nexperia.com/documents/data-sheet/74HC_HCT595.pdf, 20 Mars 2024, Nexperia B.V.
- [4] *Getting started with STM32CubeL4 MCU Package for STM32L4 series and STM32L4+ series*, Rev 16, Link: https://www.st.com/resource/en/user_manual/dm00157440-getting-started-with-stm32cubel4-mcu-package-for-stm32l4-series-and-stm32l4-series-stmicroelectronics.pdf, November 2025, STMicroelectronics
- [5] ST AME Support NF, *How to generate a one second interrupt using an STM32 timer*, Link: <https://community.st.com/t5/stm32-mcus/how-to-generate-a-one-second-interrupt-using-an-stm32-timer/ta-p/49858>, 2025-11-25

6 Appendix

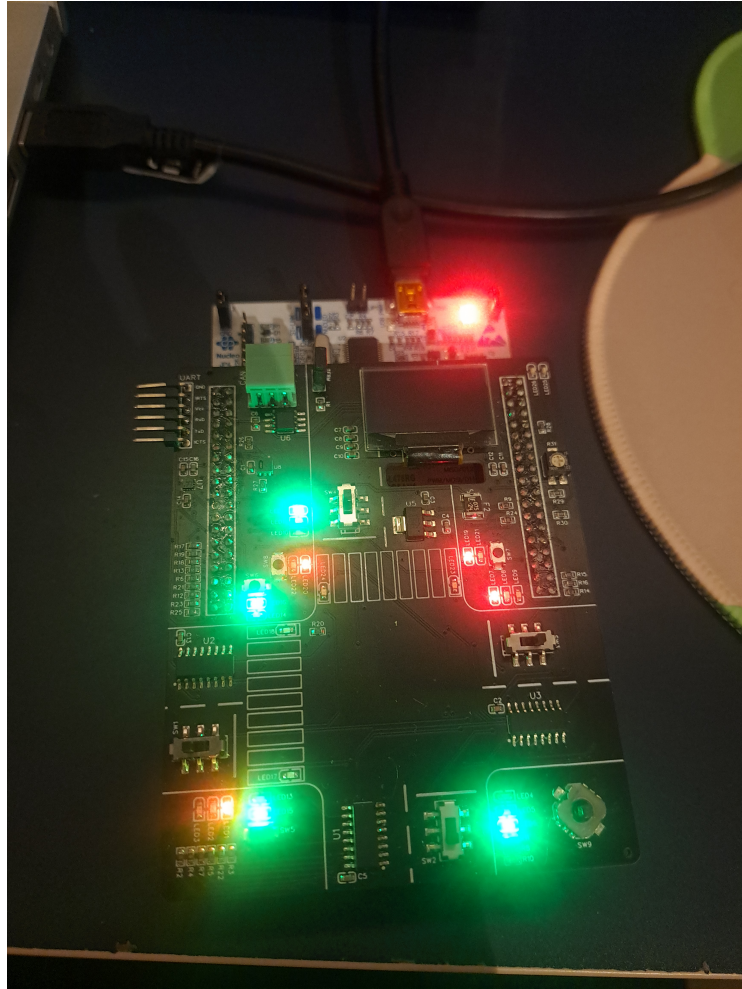


Figure 4: A Picture of the program running and managing the intersection.